

Desafio — Sistema de Contratação de Funcionários do PicPay

Spring Boot + API REST + Escolha da Tecnologia Front-end

Contexto

O PicPay está desenvolvendo um sistema interno para auxiliar o setor de Recursos Humanos no processo de contratação de novos funcionários. Sua equipe foi escolhida para desenvolver uma API capaz de cadastrar, consultar, atualizar e excluir candidatos que estão participando do processo de contratação.

Neste desafio, você deverá utilizar os conhecimentos já estudados sobre Spring Boot, métodos HTTP e ArrayList. Ao final, deverá transformar a API em um sistema web utilizando uma tecnologia Front-end de sua escolha.

Objetivo

Desenvolver um sistema web para gerenciamento de candidatos utilizando:

- Java
- Spring Boot
- Spring MVC
- ArrayList
- Uma tecnologia Front-end escolhida pelo aluno
- HTML/CSS/JavaScript, quando aplicável
- Métodos HTTP: POST, GET, PUT, PATCH e DELETE

Não é necessário utilizar banco de dados. Os candidatos deverão ser armazenados temporariamente em uma ArrayList.

1. Classe Funcionario

Crie uma classe Funcionario contendo, no mínimo, os seguintes atributos:

```
id
nome
email
telefone
cargo
departamento
salario
cidade
status
```

O atributo status poderá assumir valores como: EM_ANALISE, APROVADO, REPROVADO e CONTRATADO. Você poderá criar outros atributos caso considere necessário.

2. Armazenamento com ArrayList

Como ainda não será utilizado banco de dados, os funcionários deverão ser armazenados em uma lista:

```
ArrayList<Funcionario>
```

Ao iniciar a aplicação, você poderá cadastrar alguns funcionários fictícios para facilitar os testes.

3. Implementação dos métodos HTTP

O sistema deverá implementar os cinco principais métodos estudados durante as aulas.

Método	Endpoint	Objetivo
POST	/funcionarios	Cadastrar funcionário
GET	/funcionarios	Consultar todos
GET	/funcionarios/{id}	Consultar por ID
PUT	/funcionarios/{id}	Atualizar completamente
PATCH	/funcionarios/{id}	Atualizar parcialmente
DELETE	/funcionarios/{id}	Excluir funcionário

POST — Cadastrar funcionário

O id deverá ser único; nome, e-mail e cargo são obrigatórios; e o novo funcionário deverá ser armazenado na ArrayList.

GET — Consultar funcionários

Implemente a consulta de todos os funcionários e a consulta por ID. Caso o funcionário não exista, apresente uma mensagem adequada.

PUT — Atualizar funcionário

O PUT deverá realizar a atualização completa dos dados de um funcionário.

PATCH — Atualização parcial

O PATCH deverá permitir alterar somente alguns dados, como cargo, status ou salário, mantendo os demais atributos inalterados.

DELETE — Excluir funcionário

O método deverá localizar o funcionário pelo ID e removê-lo da ArrayList. Caso o ID não exista, informe que o funcionário não foi encontrado.

4. DESAFIO EXTRA — Escolha da tecnologia Front-end

Até este momento, os métodos da API foram testados utilizando Postman, Insomnia ou similares. Agora começa o desafio: transformar a API desenvolvida em um sistema web.

A escolha da tecnologia Front-end é livre. Você deverá selecionar uma das opções abaixo ou outra tecnologia previamente aprovada pelo professor.

Thymeleaf

Vantagens: Integração simples com Spring Boot; não exige projeto Front-end separado; utiliza HTML; boa opção para iniciantes.

Desvantagens: Menos adequado para aplicações altamente interativas; normalmente trabalha com carregamento de páginas; maior dependência entre Front-end e Back-end.

HTML + CSS + JavaScript

Vantagens: Não exige framework; excelente para compreender APIs e requisições HTTP; utiliza tecnologias fundamentais da Web.

Desvantagens: Aplicações grandes podem ficar trabalhosas; exige controle manual da interface; pode exigir mais código JavaScript.

React

Vantagens: Muito utilizado no mercado; componentização; grande ecossistema; excelente para interfaces interativas e SPA.

Desvantagens: Curva de aprendizagem maior; exige JavaScript; normalmente utiliza projeto Front-end separado; requer compreensão da comunicação entre aplicações.

Angular

Vantagens: Framework completo; estrutura bem definida; bom para aplicações grandes e ambientes corporativos.

Desvantagens: Curva de aprendizagem alta; exige TypeScript; pode ser complexo para iniciantes e para CRUDs simples.

Vue.js

Vantagens: Sintaxe relativamente simples; boa integração com HTML; interfaces reativas; organização por componentes.

Desvantagens: Exige JavaScript e aprendizado do framework; normalmente utiliza projeto separado; pode adicionar complexidade em projetos pequenos.

5. Como escolher?

Tecnologia	Dificuldade	Indicação
Thymeleaf	Baixa	Primeiro projeto Web com Java
HTML + JS	Baixa	Aprender APIs e HTTP
React	Média	Aplicações modernas
Vue.js	Média	Interfaces modernas

Tecnologia	Dificuldade	Indicação
Angular	Alta	Aplicações corporativas

6. Requisitos da interface

Independentemente da tecnologia escolhida, o sistema deverá permitir ao usuário:

- Cadastrar funcionário
- Consultar funcionários
- Consultar funcionário por ID
- Editar funcionário
- Realizar atualização parcial
- Excluir funcionário
- Pesquisar funcionários
- Visualizar os dados de maneira organizada

A interface deverá demonstrar os cinco conceitos estudados:

```
POST → criar
GET → consultar
PUT → atualizar completamente
PATCH → atualizar parcialmente
DELETE → excluir
```

7. Comunicação entre Front-end e Back-end

Se escolher HTML + JavaScript, React, Vue ou Angular, a comunicação deverá ocorrer por requisições HTTP à API Spring Boot. Você poderá utilizar Fetch API, Axios ou o mecanismo HTTP disponível na tecnologia escolhida.

Exemplo conceitual:

```
Front-end → HTTP → Spring Boot REST API → ArrayList
```

Se escolher Thymeleaf, a integração poderá ser realizada diretamente pelo Spring Boot, utilizando páginas HTML dinâmicas.

8. Busca e indicadores — Desafio adicional

Implemente uma área de busca por nome, cargo ou status. O sistema deverá apresentar somente os funcionários correspondentes aos critérios informados.

Outro diferencial será criar uma página com indicadores: Total de candidatos; Em análise; Aprovados; Reprovados; Contratados.

9. Apresentação e justificativa da escolha

Durante a apresentação, você deverá explicar:

1. Qual tecnologia Front-end você escolheu?
2. Por que escolheu essa tecnologia?
3. Como o Front-end se comunica com o Spring Boot?
4. Como realiza uma requisição GET?
5. Como realiza um POST?
6. Como realiza um PUT?
7. Como realiza um PATCH?
8. Como realiza um DELETE?
9. Quais foram as principais dificuldades encontradas?
10. Quais são as vantagens e limitações da tecnologia escolhida?

10. Diferencial visual

Você poderá utilizar Bootstrap, Tailwind CSS, Material UI ou outra biblioteca de estilização para melhorar a aparência da aplicação.

O diferencial não será simplesmente utilizar uma tecnologia mais complexa, mas conseguir explicar e demonstrar como ela se comunica com a API Spring Boot.

11. Entrega

O projeto deverá ser entregue contendo:

- Projeto Spring Boot funcionando
- Classe Funcionario
- ArrayList para armazenamento
- POST implementado
- GET implementado
- PUT implementado
- PATCH implementado
- DELETE implementado
- Controller
- Interface Front-end
- Formulário de cadastro
- Tela de listagem
- Tela de edição
- Funcionalidade de exclusão
- Funcionalidade de atualização parcial

Desafio final

O objetivo não é apenas fazer o CRUD funcionar. O desafio é transformar uma API que inicialmente funciona apenas através do Postman em um sistema web que possa ser utilizado por uma pessoa sem conhecimento técnico.

Você já conhece os métodos HTTP. Agora deverá escolher a tecnologia Front-end que considera mais adequada para construir a interface e demonstrar como ela se comunica com o Spring Boot.